

UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

Especificação do Projeto Prático Unificado
Disciplinas: Linguagem de Programação e Laboratório
de Programação

Opções de Desenvolvimento: Jogo Drifts ou Jogo Bejeweled

Prof. Thiago Soares Marques

2026

1 Informações Iniciais e Organização

O objetivo deste documento é apresentar as diretrizes para o desenvolvimento do projeto prático final das disciplinas de **Linguagem de Programação** e **Laboratório de Programação** da UEPB. O mesmo projeto será utilizado para a avaliação de ambas as disciplinas.

- **Equipes:** O projeto deverá ser desenvolvido obrigatoriamente em grupos de exatamente **4 pessoas**.
- **Linguagem:** O código fonte deve ser escrito estritamente na linguagem **C**.
- **Biblioteca Gráfica:** No lugar de outras ferramentas tradicionais, os grupos devem obrigatoriamente implementar a interface gráfica, sons e entradas utilizando a biblioteca **raylib**. Uma referência completa e documentação podem ser encontradas na página oficial: <https://www.raylib.com>.
- **Escala de Notas:** A nota do projeto variará de 0.0 a 10.0, baseando-se no cumprimento dos requisitos e na qualidade da engenharia de software empregada.

2 Diretrizes de Avaliação

Cada grupo terá um tempo estimado de 30 minutos para realizar a sua apresentação prática e defesa do código. Nesta apresentação, todos os componentes do grupo deverão estar presentes e aptos a responder de maneira individual a questões sobre toda a implementação da arquitetura do jogo.

Atenção: Não será dada uma única nota global e idêntica para o grupo. Cada componente receberá uma nota individualizada de acordo com seu desempenho, domínio do código fonte exposto e respostas durante a arguição técnica.

O software será avaliado como um ecossistema completo. A ausência de um ou mais requisitos obrigatórios acarretará perda automática de pontos, que poderá ser atenuada (embora nunca totalmente compensada) caso os componentes entregues apresentem excelente padrão de organização, legibilidade e robustez.

3 OPÇÃO DE PROJETO 1: Jogo Drifts

A primeira opção consiste no desenvolvimento do jogo de agilidade e encadeamento lógico conhecido como *Drifts*.



Figura 1: Exemplo do Jogo Drifts

3.1 Descrição e Objetivo Dinâmico

O usuário interage com o ambiente do jogo controlando, através do movimento do **mouse**, uma bola amarela principal em uma área delimitada da janela. O objetivo central é gerenciar dinamicamente uma sequência de esferas com as seguintes regras de colisão e mecânica:

- **Bolas Verdes:** Quando a bola amarela (ou o final da cauda do jogador) toca uma bola verde espalhada na tela, esta bola deve ser “colada” e inserida em uma estrutura de dados de lista que vai crescendo sequencialmente.
- **Bolas Azuis:** Tocar uma bola azul transforma as bolas verdes acumuladas na cauda do jogador em pontuação efetiva, fazendo-as desaparecer. **Requisito:** É necessário estar carregando uma cauda de pelo menos três bolas verdes para poder converter em pontos ao tocar na esfera azul.
- **Bolas Púrpuras:** Devem ser estritamente evitadas pelo jogador. Se qualquer parte da lista de bolas verdes controladas ou a bola amarela principal colidir com uma esfera púrpura, o jogador perde uma vida (ou o jogo é encerrado caso o contador de vidas chegue a zero).

3.2 Regras de Pontuação e Ordenação

Cada bola verde gerada aleatoriamente na tela possui um valor numérico atribuído. Agrupar três bolas verdes na cauda garante uma pontuação base. No entanto, bônus multiplicadores devem ser aplicados dependendo da ordenação interna da lista encadeada no momento do resgate na bola azul:

- Se as bolas coletadas estiverem organizadas estritamente em **ordem crescente** de valores (ex: 1, 2, 3, 5), aplica-se um multiplicador sobre os pontos base.
- Se os valores das bolas verdes formarem uma **progressão/sequência exata** (ex: 1, 3, 5, 7), uma bonificação extra máxima é concedida.
- *Estratégia do Jogador:* Diante de uma bola verde no cenário, o jogador deve tentar aproximar-se dela de modo a inseri-la de forma ordenada na cauda para otimizar seus pontos.

3.3 Estrutura de Telas e Interface

O jogo deverá possuir um fluxo estrutural amigável através das seguintes interfaces gráficas criadas com a raylib:

1. **Tela Inicial:** Exibe o nome do jogo, o nome dos 4 integrantes da equipe, e links de navegação para as demais telas.
2. **Tela de Instruções:** Contém as explicações detalhadas de pontuação e controles.
3. **Tela de Jogo:** Campo dinâmico onde ocorre a partida. Deve exibir na parte superior o *score* atual, as vidas restantes, o nível/fase vigente e um botão funcional de pausa.
4. **Tela Final / Game Over:** Apresenta o resultado do turno, exibe os recordes históricos salvos e permite reiniciar a gameplay sem encerrar a aplicação.

4 OPÇÃO DE PROJETO 2: Jogo Bejeweled

A segunda opção consiste na modelagem e desenvolvimento do clássico jogo de lógica e alinhamento matricial baseado em blocos/gemas conhecido como *Bejeweled*.



Figura 2: Exemplo do Jogo Bejeweled

4.1 Descrição e Mecânica Matricial

O ambiente lógico do jogo se desenvolve em uma estrutura de matriz quadrada de tamanho fixo ou configurável $N \times N$.

- **Estado Inicial:** A matriz é preenchida por texturas ou formas de gemas preciosas distintas de modo aleatório. **Restrição Crítica:** No momento em que a tela de jogo abre, o tabuleiro gerado não pode conter nenhum grupo prévio de três ou mais blocos idênticos alinhados na horizontal ou vertical.
- **Controle e Entrada:** O jogador utiliza cliques do **mouse** para selecionar e permutar a posição de duas gemas adjacentes na matriz.
- **Condições de Troca:** Uma permuta entre duas células só será validada pelo sistema se:
 1. As duas células selecionadas forem vizinhas ortogonais diretas (imediatamente acima, abaixo, à esquerda ou à direita).
 2. A troca obrigatoriamente resultar na formação de pelo menos um alinhamento de 3 ou mais gemas idênticas. Caso contrário, o movimento é considerado inválido e desfeito.
- **Mecânica de Cascata (Queda):** Quando um alinhamento válido é detectado, as gemas envolvidas desaparecem da matriz (gerando posições nulas ou vazias) e o jogador ganha pontos (5 pontos por bloco eliminado). As gemas localizadas nas

linhas superiores devem “cair” gravitariamente preenchendo as lacunas. Novas gemas devem ser sorteadas e inseridas no topo para recompor a matriz $N \times N$.

- **Reações em Cadeia:** Se o rearranjo decorrente da queda de novos blocos gerar de forma automática novos alinhamentos de 3 ou mais itens idênticos, esses grupos também devem ser eliminados sequencialmente em cadeia até que a matriz atinja estabilidade total.

4.2 Recursos Avançados Requeridos

Para uma pontuação de excelência no projeto, devem ser modelados em C os seguintes algoritmos adicionais:

- **Botão de Dica (Hint):** Sistema que analisa a matriz atual e destaca visualmente para o usuário uma combinação válida de troca existente no tabuleiro (Custo fixo: 10 pontos deduzidos do score do jogador).
- **Mecanismo de Desfazer (Undo):** Permite restaurar o estado da matriz imediatamente anterior à última jogada efetuada pelo jogador, aplicando uma penalização justa de pontuação.
- **Fim de Jogo Automático:** O programa deve realizar uma varredura matricial completa após cada ciclo de estabilização. Se não houver mais nenhuma possibilidade de movimento válido que gere um trio de gemas, o jogo deve detectar a condição de *Game Over* e transicionar para a tela final.

4.3 Estrutura de Telas

Seguindo o fluxo padrão proposto, o jogo deve conter:

1. **Tela Inicial:** Identificação do jogo, componentes da equipe e menu de navegação.
2. **Tela de Jogo Principal:** Exibição gráfica da matriz $N \times N$, exibição em tempo real do Score, indicador de tempo restante (caso implementado por tempo limitado) e botão para acionamento de dicas.
3. **Tela de Configurações:** Ajuste de parâmetros (ex: habilitar/desabilitar efeitos sonoros).
4. **Tela Final:** Exibição do placar e parabenização caso o recorde seja superado.

5 Resultados Esperados e Entregáveis

No dia agendado para a defesa prática, cada equipe deve disponibilizar e submeter os seguintes artefatos para fins de avaliação:

1. **Código Fonte Modularizado:** Todo o sistema deve ser escrito em C, fazendo uso correto de registros (**structs**), vetores, ponteiros ou matrizes dinâmicas. O código deve ser organizado de forma modular, dividindo responsabilidades lógicas e de renderização em múltiplos arquivos fonte (**.c**) e cabeçalhos (**.h**).
2. **Gerenciamento de Recursos da Raylib:** Critério rigoroso de avaliação em Laboratório de Programação. Todo recurso de mídia alocado na memória gráfica (como texturas carregadas com **LoadTexture** ou sons com **LoadSound**) deve ser explicitamente liberado ao final da execução do programa utilizando as respectivas funções de limpeza da raylib (**UnloadTexture**, **UnloadSound**, **CloseWindow**).
3. **Scripts de Compilação e README:** Junto aos arquivos fonte, deve ser disponibilizado um arquivo explicativo **README.md** detalhando todas as instruções de como vincular as diretivas da biblioteca raylib e efetuar a compilação limpa do executável nos sistemas operacionais suportados.
4. **Manual do Usuário e do Programador:** Um documento textual enxuto contendo o guia de instalação, instruções sobre como jogar o artefato desenvolvido e uma síntese de arquitetura descrevendo como o fluxo interno de dados foi modelado pela equipe.

6 Dicas Práticas para o Desenvolvimento

- **Planejamento de Ciclos:** Não tentem implementar toda a lógica gráfica de uma só vez. Foquem em estabelecer primeiro o fluxo de dados em C (como a lógica matemática de movimento de uma cauda ou a busca por padrões de trios em uma matriz) e realizem testes textuais via console antes de partir para a renderização final na raylib.
- **Controle de Versão:** O uso de ferramentas de versionamento de código (como o Git) é altamente recomendado para a divisão equilibrada das tarefas entre os 4 componentes e para assegurar pontos estáveis de recuperação em caso de erros fatais de implementação.